Assignment Title

Coursework

Coursework Type

Individual Assignment

Module Name

ST5008CEM – Programming for Developers

Intake

March

Submitted By

Name: Dipendra Kumar Sah
CU ID: 15372507
SU ID: 240151

Submitted To

Shrawan Thakur

Softwarica College of IT and E-Commerce
(Affiliated with Coventry University)

Contents

1 Question 1(A) **2**
1.1 Problem Description 2
1.2 Concept Explanation 2
1.3 Algorithm Steps 3

2 Question 1(B) **5**

3 Question 2 **8**

4 Question 3 **11**

5 Question 4 **14**

6 Question 5 **17**

7 Question 6 **20**

8 Question 7 **24**

1 Question 1(A)

Question 1 [5 Marks]

1.1 Problem Description

You are provided with a set of customer house locations, where each location is defined as a point in a two-dimensional coordinate system:

$$\texttt{customer_locations[i] = (x}_i, y_i)$$

The objective is to find the **maximum number of customer houses that lie on a single straight line**. This value represents the maximum coverage achievable using **one line-of-sight signal repeater**.

1.2 Concept Explanation

In a 2D plane, a straight line can be characterized by its **slope**. If multiple points produce the same slope when compared with a fixed reference point, those points must lie on the same straight line.

To avoid inaccuracies caused by floating-point calculations:

- Slopes are stored as **integer pairs (y, x)**.
- Each slope pair is **reduced to its simplest form** using the **Greatest Common Divisor (GCD)**.

For every point:

- Treat it as a reference.
- Compare it with all other points.
- Count how many times each normalized slope occurs.
- The slope with the highest frequency indicates the largest group of collinear points through that reference.

The global maximum across all reference points gives the final result.

1.3 Algorithm Steps

1. If the total number of points is **0, 1, or 2**, return the number of points directly (all points are collinear by definition).
2. For each point in the list:
   - Initialize a map (dictionary) to track slope frequencies.
   - Compare the current point with every other point.
   - Compute the differences in x and y coordinates.
   - Normalize the slope using GCD.
   - Update the slope count in the map.
3. Track the maximum number of points found on a single line.
4. Return the maximum count.

Python Implementation

```
from math import gcd
from collections import defaultdict

def maxPoints(customer_locations):
    n = len(customer_locations)
    if n <= 2:
        return n

    max_points = 0

    for i in range(n):
        slopes = defaultdict(int)
        x1, y1 = customer_locations[i]

        for j in range(i + 1, n):
            x2, y2 = customer_locations[j]
            dx = x2 - x1
            dy = y2 - y1

            g = gcd(dx, dy)
            dx //= g
            dy //= g

            slopes[(dy, dx)] += 1

        current_max = 1
        for count in slopes.values():
            current_max = max(current_max, count + 1)
```

```
        max_points = max(max_points, current_max)

    return max_points
```

Examples

Example 1

Input:

$$[[1,1],[2,2],[3,3]]$$

Output:

$$3$$

Example 2

Input:

$$[[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]$$

Output:

$$4$$

Complexity Analysis

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(n)$

2 Question 1(B)

Question (b) [5 Marks]

Problem Statement

You are given a continuous user search string `user_query` and a list of valid marketing keywords called `marketing_keywords_dictionary`. The task is to insert spaces into the search string such that every word formed exists in the dictionary.

A keyword from the dictionary may be reused multiple times.

The goal is to generate **all possible valid sentences**.

Approach

This problem is solved using **recursive backtracking with memoization**, commonly referred to as the *Word Break II* problem.

- The string is processed from left to right.
- At each index, all possible prefixes are checked.
- If a prefix exists in the dictionary, recursion continues for the remaining string.
- Memoization is used to store intermediate results and avoid repeated computations.
- When the end of the string is reached, a valid sentence is formed.

Algorithm

1. Convert the marketing keyword list into a set for fast lookup.
2. Define a recursive function `dfs(start)`:
   - If `start` reaches the end of the string, return an empty sentence.
   - Try all substrings starting from `start`.
   - If a substring exists in the dictionary, recursively solve the remaining string.
   - Combine the current word with results from recursion.
3. Store results in a memo dictionary.
4. Return all valid keyword segmentations.

Python Implementation

```
def keywordSegmentation(user_query, marketing_keywords_dictionary):
    word_set = set(marketing_keywords_dictionary)
    memo = {}

    def dfs(start):
        if start in memo:
            return memo[start]

        if start == len(user_query):
            return [""]

        sentences = []

        for end in range(start + 1, len(user_query) + 1):
            word = user_query[start:end]
            if word in word_set:
                for sub in dfs(end):
                    if sub:
                        sentences.append(word + "␣" + sub)
                    else:
                        sentences.append(word)

        memo[start] = sentences
        return sentences

    return dfs(0)
```

Examples

Example 1

Input:

- user_query = "nepaltrekkingguide"
- dictionary = {"nepal", "trekking", "guide", "nepaltrekking"}

Output:

- nepal trekking guide
- nepaltrekking guide

Example 2

Input:

- user_query = "visitkathmandunepal"

Output:

- visit kathmandu nepal
- visitkathmandu nepal
- visit kathmandunepal

Example 3
Input:

- user_query = `"everesthikingtrail"`

Output:

- No valid segmentation

Diagram Illustration

Example 1: `nepaltrekkingguide`

```
|-- Node A
|-- Node B
`-- Node C
```

Complexity Analysis

- **Time Complexity:** Exponential in the worst case, reduced significantly using memoization.
- **Space Complexity:** $O(n \times k)$

3 Question 2

Question 2: Hydropower Plant Cascade Efficiency [5 Marks]

Context

You are an energy consultant evaluating a cascade of hydropower plants along a river system in Nepal. Each plant contributes a net power value:

- Positive value → Power generation
- Negative value → Environmental or economic cost

The river system forms a tree structure, where upstream plants can contribute power to downstream plants.

Problem Statement

Given the root of a binary tree, where `root.val` represents the net power generation (or cost) at a hydropower plant, determine the **maximum total net power generation** obtainable from any continuous sequence of connected plants (any path in the tree).

Constraints

- The path can start and end at any node
- Each plant can be used only once
- The path must follow parent–child connections

Explanation

This is a classic **Maximum Path Sum in a Binary Tree** problem.

A valid path may include:

- Only the current node
- Current node + left subtree
- Current node + right subtree
- Left subtree + current node + right subtree

If a subtree contributes a negative value, it is excluded to maximize total power.

Algorithm

1. Initialize a global variable `max_sum` to negative infinity.
2. Define a recursive function `maxGain(node)`:
   - If the node is `None`, return 0.
   - Compute maximum gain from left and right subtrees.
   - Ignore negative gains using `max(0, gain)`.
   - Calculate current path sum:

     `node.val + left_gain + right_gain`

   - Update `max_sum`.
   - Return the maximum gain including the current node.
3. Call `maxGain(root)`.
4. Return `max_sum`.

Python Implementation

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def maxPathSum(root):
    max_sum = float('-inf')

    def maxGain(node):
        nonlocal max_sum
        if not node:
            return 0

        left_gain = max(maxGain(node.left), 0)
        right_gain = max(maxGain(node.right), 0)

        current_path = node.val + left_gain + right_gain
        max_sum = max(max_sum, current_path)

        return node.val + max(left_gain, right_gain)

    maxGain(root)
    return max_sum
```

Examples

Example 1

```
    1
   / \
  2   3
```

Optimal Path:

$$2 \to 1 \to 3$$

Total Power:

$$2 + 1 + 3 = 6$$

Output: 6

Example 2

```
     -10
     /  \
    9    20
        /  \
       15    7
```

Optimal Path:

$$15 \to 20 \to 7$$

Total Power:

$$15 + 20 + 7 = 42$$

The high-cost site `-10` is excluded.
Output: 42

Complexity Analysis

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(h)$

4 Question 3

Question 3: Agricultural Commodity Trading [5 Marks]

Context

You are a commodities trader in Chitwan, Nepal, dealing with seasonal agricultural products such as rice, maize, and lentils. Commodity prices fluctuate daily due to harvest cycles, demand, festivals, and transportation conditions. Due to limited capital, you are allowed to perform only a limited number of buy–sell transactions.

Problem Statement

Given:

- An integer `max_trades` representing the maximum number of transactions allowed
- An array `daily_prices`, where `daily_prices[i]` is the price on day i

Determine the **maximum profit** that can be achieved under the following constraints:

- At most `max_trades` transactions are allowed
- You must sell before buying again
- Buying and selling on the same day is not allowed

Explanation

This is the classic **Stock Trading with at Most K Transactions** problem.

Key observations:

- Each transaction consists of one buy followed by one sell
- If `max_trades` is large compared to the number of days, the problem reduces to unlimited transactions
- Otherwise, Dynamic Programming is used to compute the optimal profit

Dynamic Programming Approach

DP Definition:

$$dp[t][i] = \text{maximum profit using at most } t \text{ transactions up to day } i$$

State Transition:

$$dp[t][i] = \max\left(dp[t][i-1],\ \texttt{daily_prices}[i] + \text{maxDiff}\right)$$

Where:

$$\text{maxDiff} = \max\left(\text{maxDiff},\ dp[t-1][i] - \texttt{daily_prices}[i]\right)$$

Algorithm

1. If the prices array is empty or `max_trades = 0`, return 0
2. If `max_trades` $\geq \frac{n}{2}$, perform greedy unlimited transactions
3. Initialize a DP table of size $(\texttt{max_trades} + 1) \times n$
4. Fill the DP table iteratively using the transition formula
5. Return `dp[max_trades][n-1]`

Python Implementation

```
def maxProfit(max_trades, daily_prices):
    n = len(daily_prices)
    if n == 0 or max_trades == 0:
        return 0

    if max_trades >= n // 2:
        profit = 0
        for i in range(1, n):
            if daily_prices[i] > daily_prices[i - 1]:
                profit += daily_prices[i] - daily_prices[i - 1]
        return profit

    dp = [[0] * n for _ in range(max_trades + 1)]

    for t in range(1, max_trades + 1):
        max_diff = -daily_prices[0]
        for i in range(1, n):
            dp[t][i] = max(dp[t][i - 1], daily_prices[i] + max_diff)
            max_diff = max(max_diff, dp[t - 1][i] - daily_prices[i])

    return dp[max_trades][n - 1]
```

Example

Input

- max_trades = 2
- daily_prices = [2000, 4000, 1000]

```
Day:     1     2     3
Price: 2000 → 4000 → 1000
```

Output: 2000

Complexity Analysis

- **Time Complexity:** $O(\texttt{max_trades} \times n)$
- **Space Complexity:** $O(\texttt{max_trades} \times n)$

5 Question 4

Question 4: Smart Energy Grid Load Distribution Optimization (Nepal)

[Total Marks: 20]

Introduction

As Nepal transitions toward sustainable energy, smart grids are essential for dynamically distributing electricity from renewable and non-renewable sources. This problem focuses on optimizing hourly electricity distribution across multiple districts while minimizing cost and reducing diesel usage, under realistic constraints such as capacity limits and availability windows.

The solution combines:

- **Dynamic Programming (DP)** for feasibility under constraints
- **Greedy Strategy** for cost minimization

1. Modeling the Input Data [2 Marks]

Energy demand and supply sources are modeled using Python dictionaries and lists.

Hourly Demand (kWh)

```
hourly_demand = {
    6: {"A": 20, "B": 15, "C": 25},
    7: {"A": 22, "B": 16, "C": 28}
}
```

Energy Sources

```
energy_sources = [
    {"id": "S1", "type": "Solar", "capacity": 50, "start": 6, "end":
        18, "cost": 1.0},
    {"id": "S2", "type": "Hydro", "capacity": 40, "start": 0, "end":
        24, "cost": 1.5},
    {"id": "S3", "type": "Diesel", "capacity": 60, "start": 17, "end
       ": 23, "cost": 3.0}
]
```

2. Hourly Allocation using Dynamic Programming [5 Marks]

Approach

For each hour:

- Compute total district demand
- Identify available energy sources
- Ensure:
  - Demand is met within ±10%
  - Source capacity is not exceeded
  - All districts receive power

Dynamic Programming ensures feasibility, while greedy allocation minimizes cost.

3. Greedy Source Prioritization [3 Marks]

Strategy

Available energy sources are sorted by cost per kWh:

```
available_sources.sort(key=lambda x: x["cost"])
```

Priority order:

- Solar (cheapest)
- Hydro
- Diesel (last resort)

This minimizes cost and reduces carbon emissions.

4. Approximate Demand Satisfaction (±10%) [3 Marks]

```
min_demand = demand * 0.9
max_demand = demand * 1.1
```

5. Output Table of Results [3 Marks]

Sample Output for Hour 06

District	Solar	Hydro	Diesel	Demand	% Fulfilled
A	20	0	0	20	100%
B	15	0	0	15	100%
C	15	10	0	25	100%

6. Descriptive Calculation: Hour 06

$$20 + 15 + 25 = 60 \text{ kWh}$$

7. Cost and Resource Usage Analysis [4 Marks]

$$\text{Solar: } 50 \times 1.0 = Rs.\ 50$$

$$\text{Hydro: } 10 \times 1.5 = Rs.\ 15$$

$$\textbf{Total Cost = Rs. 65}$$

Algorithm Efficiency and Trade-offs

- **Time Complexity:** $O(H \times S \times D)$
- **Advantages:** Renewable-first, cost-efficient
- **Trade-offs:** Greedy may miss global optimum

6 Question 5

Tourist Spot Optimizer – GUI with Heuristic-Based Itinerary Planner

[Total Marks: 20]

Problem Statement

Tourists visiting Nepal often face constraints such as limited time, limited budget, and specific interests (nature, culture, adventure). The objective of this system is to design a **GUI-based itinerary planner** that generates an optimal or near-optimal travel path among tourist destinations using a **heuristic optimization algorithm**.

The system:

- Takes user preferences via a GUI
- Uses a greedy heuristic to build an itinerary
- Visualizes the travel path
- Compares heuristic results with a brute-force optimal solution

1. GUI Design for User Input [3 Marks]

Design Features

The graphical user interface collects:

- Total available time (hours)
- Maximum budget (Nepali Rupees)
- Interest tags (Nature, Culture, Adventure)
- Button to generate itinerary
- Output section for results and explanation

Technology Used

- **Python Tkinter** for GUI
- **Matplotlib** for path visualization

2. Tourist Spot Dataset [3 Marks]

```
tourist_spots = [
    {"name": "Pashupatinath␣Temple", "lat": 27.7104, "lon": 85.3488,
     "fee": 100, "time": 1.5, "tags": ["culture", "religious"]},
    {"name": "Swayambhunath␣Stupa", "lat": 27.7149, "lon": 85.2906,
     "fee": 200, "time": 1.5, "tags": ["culture", "heritage"]},
    {"name": "Garden␣of␣Dreams", "lat": 27.7125, "lon": 85.3170,
     "fee": 150, "time": 1.0, "tags": ["nature", "relaxation"]},
    {"name": "Chandragiri␣Hills", "lat": 27.6616, "lon": 85.2458,
     "fee": 700, "time": 3.0, "tags": ["nature", "adventure"]},
    {"name": "Kathmandu␣Durbar␣Square", "lat": 27.7048, "lon":
       85.3076,
     "fee": 100, "time": 2.0, "tags": ["culture", "heritage"]}
]
```

3. Heuristic Optimization Algorithm (Greedy) [5 Marks]

$$\text{Score} = \frac{\text{Interest Match Count}}{\text{Entry Fee} + 1}$$

```
def score(spot, interests):
    match = len(set(spot["tags"]) & interests)
    return match / (spot["fee"] + 1)
```

4. Itinerary Display and Map Visualization [5 Marks]

```
import matplotlib.pyplot as plt

def plot_path(itinerary):
    x = [s["lon"] for s in itinerary]
    y = [s["lat"] for s in itinerary]

    plt.plot(x, y, marker='o')
    for s in itinerary:
        plt.text(s["lon"], s["lat"], s["name"], fontsize=8)

    plt.xlabel("Longitude")
    plt.ylabel("Latitude")
    plt.title("Tourist␣Itinerary␣Path")
    plt.show()
```

5. Brute-Force Comparison [4 Marks]

```
import itertools

def brute_force(spots, max_time, max_budget):
    best = []
    for r in range(1, len(spots)+1):
        for perm in itertools.permutations(spots, r):
            time = sum(s["time"] for s in perm)
            cost = sum(s["fee"] for s in perm)
            if time <= max_time and cost <= max_budget:
                if len(perm) > len(best):
                    best = perm
    return best
```

6. GUI Integration (Tkinter Skeleton)

import tkinter as tk
root = tk.Tk() root.title("Tourist Spot Optimizer")
tk.Label(root, text="Max Budget (Rs)").pack() bud

7 Question 6

Multi-threaded Weather Data Collector

[Total Marks: 20]

Problem Statement

The objective of this assignment is to design a **GUI-based weather data collector** that retrieves real-time weather data for five major cities of Nepal using a free public weather API. The system utilizes **multithreading** to fetch data concurrently while keeping the GUI responsive and compares the latency of **sequential vs parallel** data fetching.

1. GUI Design [3 Marks]

Design Requirements

- Responsive GUI (no freezing during data fetch)
- "Fetch Weather" button
- Tabular display of weather information
- Latency comparison results

Technology Used

- **Python Tkinter** – GUI development
- **threading** – Multithreading
- **requests** – API calls
- **matplotlib** – Latency visualization

2. Fetch Weather Data for Five Locations [3 Marks]

Weather API

OpenWeatherMap API (Free tier, API key required)

Cities Used

- Kathmandu
- Pokhara
- Biratnagar
- Nepalgunj
- Dhangadhi

Weather Parameters Fetched

- Temperature (°C)
- Humidity (%)
- Pressure (hPa)

3. Multithreading Implementation [4 Marks]

Approach

- One thread per city (5 threads total)
- Each thread performs an independent API call
- Results stored in a shared dictionary
- Lock used to ensure thread safety

```
import threading
import requests

def fetch_weather(city, results, lock):
    url = f"https://api.openweathermap.org/data/2.5/weather?q={city
       }&appid=YOUR_API_KEY&units=metric"
    data = requests.get(url).json()
    with lock:
        results[city] = {
            "temp": data["main"]["temp"],
            "humidity": data["main"]["humidity"],
            "pressure": data["main"]["pressure"]
        }
```

4. Thread-Safe GUI Updates [5 Marks]

Tkinter is **not thread-safe**. GUI updates are executed using the `after()` method while shared data is protected using `threading.Lock`.

```
lock = threading.Lock()
results = {}

def update_table():
    for city, data in results.items():
        table.insert("", "end",
        values=(city, data["temp"], data["humidity"], data["pressure
           "]))
```

5. Sequential vs Multithreaded Latency Comparison [5 Marks]

Sequential Fetching

```
import time

def fetch_sequential(cities):
    start = time.time()
    for city in cities:
        fetch_weather(city, {}, lock)
    return time.time() - start
```

Parallel Fetching

```
def fetch_parallel(cities):
    start = time.time()
    threads = []
    for city in cities:
        t = threading.Thread(
            target=fetch_weather,
            args=(city, results, lock))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    return time.time() - start
```

6. Complete GUI Skeleton (Tkinter)

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Multi-threaded␣Weather␣Data␣Collector")

columns = ("City", "Temp␣( C )", "Humidity␣(%)", "Pressure␣(hPa)")
table = ttk.Treeview(root, columns=columns, show="headings")

for col in columns:
    table.heading(col, text=col)

table.pack()

tk.Button(root, text="Fetch␣Weather").pack()
root.mainloop()
```

Final Evaluation Summary

Task	Marks
GUI Design	3
API Data Fetching	3
Multithreading	4
Thread Safety	5
Latency Comparison	5
Total	**20 / 20**

8 Question 7

Part A: Scenario and Network Model

Scenario

A major 7.8 magnitude earthquake struck central Nepal, epicenter near Gorkha district. Kathmandu and surrounding cities in Bagmati Province suffered significant damage. Roads cracked, bridges were compromised, and aftershocks continue.

As a logistics expert for NDRRMA, the task is to design algorithmic models for efficient emergency supply distribution.

Locations and Roles

Network Graph Details

- Directed graph $G = (V, E)$ models the roads.
- Each edge (u, v) has a safety probability $0 < p(u, v) \leq 1$.
- Each edge (u, v) has a capacity $c(u, v)$ (max trucks/hour).

Part B: Assignment Questions

Question 1: Problem Modeling [4 Marks]

(a) Why Standard Dijkstra is Not Suitable [2 Points]

Safety of a path is defined as:

$$\text{Safety(path)} = \prod_{e \in \text{path}} p(e)$$

Dijkstra minimizes the sum of edge weights. Safety probabilities are **multiplicative**, not additive. **Conclusion:** Standard Dijkstra cannot model path safety directly.

(b) Why Maximizing Probabilities Directly Fails [2 Points]

Dijkstra assumes monotonicity: once a node's shortest path is found, it cannot improve. Multiplying probabilities $0 < p < 1$ decreases the path product with each edge. **Conclusion:** Direct maximization of probabilities violates Dijkstra's greedy correctness.

Question 2: Algorithm Adaptation for Safest Path [9 Marks]

(a) Weight Transformation Using Logarithms [3 Points]

Transform each edge e with probability $p(e)$:

$$w(e) = -\log(p(e))$$

Then maximizing $\prod p(e)$ becomes equivalent to minimizing $\sum -\log(p(e))$. All weights are non-negative $\rightarrow$ Dijkstra assumptions are satisfied.

(b) Adapted Dijkstra Implementation [3 Points]

```
import math, heapq

def safest_path(graph, source):
    # graph[u] = list of tuples (v, p(u,v))
    dist = {v: float('inf') for v in graph}
    dist[source] = 0
    parent = {v: None for v in graph}

    pq = [(0, source)]
    while pq:
        current_dist, u = heapq.heappop(pq)
        if current_dist > dist[u]:
            continue
        for v, prob in graph[u]:
            weight = -math.log(prob)
            if dist[v] > dist[u] + weight:
                dist[v] = dist[u] + weight
                parent[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, parent
```

(c) Proof of Correctness [3 Points]

Maximizing safety $\Leftrightarrow$ minimizing

$$-\log\left(\prod p(e)\right) = \sum -\log(p(e))$$

All weights ≥ 0, Dijkstra assumptions hold. Therefore, the adapted algorithm finds the path with maximum safety probability.

Safety Probability Network Diagram

Question 3: Maximum Throughput Analysis [7 Marks]

(a) Max-Flow Model [2 Points]

Source (s): KTM Sink (t): BS

Edges with capacity $c(u, v)$ (trucks/hour):

(b) Edmonds-Karp Execution [3 Points]

- Iteration 1: Path KTM $\rightarrow$ JA $\rightarrow$ BS, Bottleneck = 6, Push 6 trucks

Node	Description
KTM	Kathmandu Depot (Tribhuvan Airport) – Primary supply collection
PH	Patan Hospital – Urgent medical supplies; prioritize path safety
BS	Bhaktapur Shelter – Houses thousands; prioritize maximum throughput
JA	Junction A – Major traffic intersection
JB	Junction B – Major traffic intersection

Figure 1: Safety Probability Network showing nodes and edge probabilities.

Edge	Capacity
KTM → JA	10
KTM → JB	8
JA → BS	6
JB → BS	5
JA → JB	4

- Iteration 2: Path KTM $\rightarrow$ JB $\rightarrow$ BS, Bottleneck = 5, Push 5 trucks
- Iteration 3: No augmenting path $\rightarrow$ terminate

(c) Max Flow and Min Cut [2 Points]

$$\text{Maximum flow} = 6 + 5 = 11 \text{ trucks/hour}$$

Min-cut: $S = \{KTM, JA, JB\}, T = \{BS\}$, Cut edges: JA$\rightarrow$BS=6, JB$\rightarrow$BS=5 $\rightarrow$ sum=11
Max-flow = Min-cut theorem verified

Maximum Throughput Network Diagram

Summary Table

Marks Allocation

Figure 2: Maximum Throughput Network showing source, sink, and edge capacities.

Question	Key Concept	Algorithm
Q1	Safest Path	Dijkstra with $-\log(p)$
Q2	Max-Product Safety	Transformed weights, Dijkstra
Q3	Max Throughput	Edmonds-Karp Max Flow

Task	Marks
Q1 (Problem Modeling)	4
Q2 (Safest Path)	9
Q3 (Max Throughput)	7
Total	20 / 20